



Reverse Engineering SPJ-Queries from Examples

Yaacov Y. Weiss

Rachel and Selim Benin School of Computer
Science and Engineering
Hebrew University of Jerusalem
Jerusalem, Israel
yyweiss@cs.huji.ac.il

Sara Cohen

Rachel and Selim Benin School of Computer
Science and Engineering
Hebrew University of Jerusalem
Jerusalem, Israel
sara@cs.huji.ac.il

ABSTRACT

This paper investigates the problem of *reverse engineering*, i.e., *learning*, select-project-join (SPJ) queries from a user-provided example set, containing positive and negative tuples. The goal is then to determine whether there exists a query returning all the positive tuples, but none of the negative tuples, and furthermore, to find such a query, if it exists. These are called the *satisfiability* and *learning* problems, respectively. The ability to solve these problems is an important step in simplifying the querying process for non-expert users.

This paper thoroughly investigates the satisfiability and learning problems in a variety of settings. In particular, we consider several classes of queries, which allow different combinations of the operators select, project and join. In addition, we compare the complexity of satisfiability and learning, when the query is, or is not, of bounded size. We note that bounded-size queries are of particular interest, as they can be used to avoid over-fitting (i.e., tailoring a query precisely to only the seen examples).

In order to fully understand the underlying factors which make satisfiability and learning (in)tractable, we consider different components of the problem, namely, the size of a query to be learned, the size of the schema and the number of examples. We study the complexity of our problems, when considering these as part of the input, as constants or as parameters (i.e., as in parameterized complexity analysis). Depending on the setting, the complexity of satisfiability and learning can vary significantly. Among other results, our analysis also provides new problems that are complete for $W[3]$, for which few natural problems are known. Finally, by considering a variety of settings, we derive insight on how the different facets of our problem interplay with the size of the database, thereby providing the theoretical foundations necessary for a future implementation of query learning from examples.

1. INTRODUCTION

Increasingly many datasets are publicly available, and can be freely accessed. Census data, US and European government data, geospatial datasets, bibliographic information, health-care statistics, human genetic information and NASA's database of satellite imagery of Earth, are just a few of such datasets. However, public datasets can be effectively leveraged only if users can formulate their queries of interest. Unfortunately, for naive users, formulating a query in SQL or in relational algebra can be a very challenging task.

One way to simplify the querying process is to allow users to provide a set of positive and negative examples, i.e., tuples they expect to appear in the result, and tuples they expect not to appear in the result. Then, a system could attempt to learn a query of interest to the user, which satisfies the given examples. This can be useful for users who are somewhat familiar with the query language, as the system would suggest a satisfying query, which could be adjusted by the user to derive his particular query of interest. Adjusting a given query may be easier than deriving a query from scratch. Second, and more importantly, for naive users who are not familiar with the query language at all, the system could learn a query satisfying the examples, and then present all of the query results to the user. Using an interactive process (as described in [2]), the user may mark additional tuples as positive and negative examples, thereby eventually finding data of interest. Such a system is useful even for languages for which precise learning of a concept is intractable [3, 4]. We do not describe an interactive system, rather consider the underlying single step of deriving a query from a set of examples.

This paper studies the problem of *learning*, i.e., reverse engineering, a select-project-join query satisfying a given set of positive and negative examples. To be more a bit more precise, given a database and an example set, we study the complexity of finding a query that returns all positive examples, but none of the negative examples, over the given database.¹ The complexity of this problem is highly dependent on the types of operators that are allowed in the query to be learned. An orthogonal issue is which components of the problem are part of the input, and which can be considered constants, or parameters (as in parameterized complexity analysis). We consider the size of the query to be

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

PODS'17, May 14-19, 2017, Chicago, IL, USA

© 2017 ACM. ISBN 978-1-4503-4198-1/17/05...\$15.00

DOI: <http://dx.doi.org/10.1145/3034786.3056112>

¹This is different from the learnability problem, which focuses on the complexity of learning a specific query, e.g., [1]. We are simply interested in finding a candidate query satisfying the examples.

learned (for bounded sized queries), the size of the schema and the number of examples, as possible inputs, constants or parameters to our problems.

The following examples demonstrate some of the issues that arise when learning queries.

EXAMPLE 1.1 (UNBOUNDED SIZE QUERIES). Consider the database $\mathcal{D}$ appearing in Figure 1, (while ignoring the “✓” and “✗”-marks), and an example set consisting of the positive tuple $t_p = \{A : 1, C : 2, E : 5\}$ and the negative tuple $t_n = \{A : 1, C : 2, E : 4\}$. When learning a query containing selections and projections, the query

$$\pi_{A,C,E}(\sigma_{A=1}(\sigma_{C=2}(\sigma_{E=5}(R_6))))$$

satisfies the example set over $\mathcal{D}$ as it returns t_p , but not t_n . However, this query is rather restrictive, as it can return only t_p , and no other tuples. Thus, it suffers from over-fitting of the examples, and does not help us generalize beyond the user’s examples. When bounding the size of the query to contain at most two operators, one satisfying query is

$$\pi_{A,C,E}(\sigma_{E \geq 5}(R_6)).$$

Clearly, such a query is of more interest. Hence, we will also study the problem of learning queries of bounded size. However, finding queries of bounded size will usually be more difficult (unless the bound is considered to be a constant).

EXAMPLE 1.2 (OPERATOR TYPES). Another issue that affects the complexity of learning is the combinations of operators allowed in the query to be learned. For some combinations, simply determining whether a given query satisfies the example set can be intractable. This holds, e.g., for queries containing projection and natural join. Thus, even if we can identify a small number of candidate queries that must be considered in order to satisfy an example set, it still may be intractable to determine if there exists a satisfying query. To demonstrate, for the tuples t_p and t_n in Example 1.1, the query

$$\pi_{A,C,E}(R_5 \bowtie R_6),$$

is satisfying. However, determining this in the general case is difficult.

In this case, assuming a constant bound on the query size makes learning tractable, as query evaluation, for constant sized queries, is in PTIME. In other cases, assuming a bound on the size of the schema, or the number of examples, can yield tractability. All such assumptions are natural, as the size of the query, schema, and example set, are likely to be significantly smaller than the size of the database.

EXAMPLE 1.3 (CONSTANTS VS. PARAMETERS). While the size of the query, schema, and example set are likely to be not very large, they may also not be negligible. Therefore, an algorithm running in time $\mathcal{O}(|\mathcal{D}|^k)$, where k is one of the above parameters, will probably not be efficient enough in practice. Hence, it is of great interest to consider the parameterized complexity analysis of our problems.

For example, consider the problem of learning a query of bounded size k , containing projection and selection, such as the query $\pi_{A,C,E}(\sigma_{E \geq 5}(R_6))$ from Example 1.1. This problem is in PTIME, when k is considered a constant. But, when k is a parameter to the problem, it becomes W[3]-complete. Therefore, it is unlikely for an algorithm to exist for this problem, that runs in time polynomial in $|\mathcal{D}|$ and exponential in k .

The main contribution of this paper is a full characterization of the complexity of learning a query from examples, as a function of the type of query to be learned, and the input parameters to the problem. This yields insight into how the various facets of our problem interplay with one another and provides the theoretical foundations necessary for a future implementation of query learning from examples. Among other results, our analysis also provides new problems that are complete for W[3], for which few natural problems are known.

This paper is structured as follows. In Section 2, we discuss related work on learning queries from examples. Section 3 provides necessary notation and definitions, and formally defines our problems of interest. In Section 4 we consider the complexity of learning and satisfiability when no parameters are given. In Sections 5, 6 and 7 we study learning and satisfiability when the size of the query, size of the schema, and number of examples, respectively, are parameters. Finally, in Section 8, we conclude. We note that throughout the paper, we omit formal proofs, and provide intuition and examples, instead. Due to lack of space, we include only the proofs for our W[3]-completeness results in the appendix.

2. RELATED WORK

The problem of learning a concept from a set of examples, often called the *reverse engineering problem*, has been studied for decades, e.g., [16, 19] studied the problem of learning an automaton, based on examples. Within the area of databases, [10] consider learning XPath from XML examples, [11, 12] learn tree patterns from graph examples and [8] learn path queries from graphs. Recently, [5] study the problem of reverse engineering SPARQL queries from example triples. Due to the difference in the data models and languages, the above work does not imply results on learning relational algebra from queries. We note that SPARQL does share some characteristics with relational algebra. However, the setting of [5] is very different from ours, e.g., the schema is of constant width, queries are unbounded in size, and there is no projection or natural join.

Several papers have considered learning join queries from examples. In [24], the problem of reverse engineering join queries is considered. However, they assume that they are given the complete answer, and they must find a query (without selections) returning precisely this result. In addition to differences in the problem setting, [24] deals with practical aspects, and does not study the complexity of the problems. In [23], the focus is also on finding a query returning precisely a given result (or one that is approximately the same), while [22] studies the problem of refining a given query, to include additional results. In a recent work, [7] studies the complexity of learning queries from examples. They also do not allow selections, but they allow for arbitrary equijoins (unlike our setting which includes only natural join). When arbitrary equijoins are allowed, satisfying queries can be of exponential size, which is a major source of complexity in their setting. To overcome this problem, they consider queries with bounded tree width. In our paper, we only allow for natural joins, thus avoiding exponential size queries. (This is also the reason why we do not allow renaming of attributes in our queries.) This difference implies completely different complexity results.

Learnability of conjunctive queries was the focus of [9]. Learnability differs from the problem at hand as it studies

R_1			R_2			R_3			R_6					
A	C	D	B	C	D	A	B	D	A	B	C	D	E	
1	1	1	1	1	1	1	1	1	✓	1	2	3	4	5
1	2	2	2	2	2	1	2	2	✗	1	3	2	3	4
										2	4	4	1	3
										5	3	2	4	2
									✗	4	2	3	1	2
										2	2	4	3	2
									✓	1	1	2	1	5
									✓	1	5	4	2	3

R_4			R_5		
A	C	D	A	B	C
1	1	1	1	2	1
1	1	2	1	2	2
			1	1	2

Figure 1: Database for a running example.

the complexity of the process of honing in on one specific query, while interactively receiving examples from the user, as opposed to simply finding an arbitrary candidate query satisfying all examples. (Satisfiability is also considered in [9], but there are significant differences between the query language considered there, and our language, e.g., [9] does not have selection, and has limited projection.) In [18], the focus is also in learnability, and it is shown that under cryptographic assumptions, learnability of acyclic conjunctive queries is hard.

It is also possible to view the problem at hand as a variation of the general question of the expressive power of relational algebra, studied in [6] and [21]. While several of the scenarios we consider can be formulated in these terms, the differences in the settings prevent results from [6, 21] from being applicable in our settings. Intuitively, these papers consider the following problem. Given a database $\mathcal{D}$ and a relation R , does there exist a query in relational algebra that returns precisely R from $\mathcal{D}$. They consider different operators from those considered in this paper. In addition, they aim to find a query returning a precise relation, while our requirements on the relation to be returned are different—we only require inclusion and exclusion of a set of tuples. Thus, the results of [6, 21] are not applicable in our setting. Placing our work in the wider framework of the expressive power of relational algebra can give rise to interesting future work, e.g., extensions of the style of [14] to the work of [6, 21] can be considered for the problem we investigate.

Beyond the significant differences in settings between our work and previous work, no work to date has considered the parameterized complexity of learning queries from examples. Parameterized complexity provides a fine-grained analysis, which clearly delineates which factors give rise to intractability in practice. Parameterized complexity and the W -hierarchy was first introduced in [13]. A summary of results on parameterized complexity of query evaluation appears in [17]. The main result related to our work, is that of [20], which shows (among other results) that evaluation of conjunctive queries is $W[1]$ -complete with respect to the size of the query. We utilize this result in several proofs.

3. DEFINITIONS

Relations, Databases and Schemas. A tuple is a set of pairs $t = \{A_1 : c_1, \dots, A_n : c_n\}$, with $n \geq 1$, where A_i is an attribute and c_i is a value in a totally ordered domain. In

addition, we require $A_i \neq A_j$, for all $i \neq j$. We use $ar(t)$ to denote the *arity* of t , i.e., n , and $schema(t)$ to denote the *schema* of t , i.e., $\{A_1, \dots, A_n\}$.

A relation $R, S, \dots$ is a named set of tuples, all of which have the same schema. We use $ar(R)$ and $schema(R)$ to denote the arity and schema of the tuples in R . We use $name(R)$ to denote the name of the relation R . A database is a set of relations $\mathcal{D} = \{R_1, \dots, R_n\}$. We use $schema(\mathcal{D})$ to denote the set of pairs $\{name(R_i) : schema(R_i) \mid i \leq n\}$. We use $size(schema(\mathcal{D}))$ to denote the sum of arities of all relations in $schema(\mathcal{D})$. In particular, note that if $size(schema(\mathcal{D})) = k$, then $schema(\mathcal{D})$ contains at most k relations.

EXAMPLE 3.1. The database instance $\mathcal{D}$ which appears in Figure 1 contains 6 relations, with a total of 19 tuples, and has a schema of size 20.

Queries. We use Q, Q' to denote queries. Queries will be written in relational algebra, using the operations natural join, selection and projection, denoted $\bowtie, \sigma, \pi$, respectively. For queries with disjoint schemas, natural join is simply a Cartesian product. We use Θ to denote the set of comparison operators $\{=, \neq, \leq, \geq\}$. Conditions in a selection operation (abstractly denoted as ψ) are comparisons of attributes with constants, of the form $A_i \theta c$ where A_i is an attribute, c is a constant, and $\theta \in \Theta$. Each selection operation contains a single comparison. Sometimes we will restrict the discussion to subsets of Θ , such as $\Theta_{pos} = \Theta - \{\neq\}$. Note that we only consider natural join in this paper, and not arbitrary conditional joins. In addition, to simplify, we assume that join always precedes selection, which, in turn, precedes projection.

We use $Q(\mathcal{D})$ to denote the set of tuples derived by applying query Q to database $\mathcal{D}$. We use $ar(Q)$ to denote the arity of the results of Q and $schema(Q)$ to denote the schema of the result of Q , defined recursively as follows:

1. $schema(R)$, if $Q = R$, for some relation R ;
2. $schema(Q')$, if $Q = \sigma_\psi(Q')$;
3. $\{A_1, \dots, A_k\}$, if $Q = \pi_{A_1, \dots, A_k}(Q')$;
4. $schema(Q') \cup schema(Q'')$, if $Q = Q' \bowtie Q''$.

EXAMPLE 3.2. Consider the queries $Q_1 = \pi_{A,D}(R_1 \bowtie R_6)$ and $Q_2 = \sigma_{D \geq 2}(R_6)$. Then, we have

$$schema(Q_1) = \{A, D\} \quad schema(Q_2) = \{A, B, C, D, E\}.$$

Example Sets. The result of applying a query to a database is a set of tuples. Hence, such tuples can be used as examples of what the user expects (or does not expect) to be returned by a query. Formally, an *example* e is a tuple. We say that e is a *positive example* if it is expected to appear in the output, and is a *negative example*, otherwise.

An *example set* E is a set of positive and negative examples, all having the same schema, denoted $\text{schema}(E)$. Given a database $\mathcal{D}$ and an example set E , we say that a query Q *satisfies* E if (1) $\text{schema}(E) = \text{schema}(Q)$ and (2) for all $e \in E$, it holds that $e \in Q(\mathcal{D})$ if and only if e is a positive example.

EXAMPLE 3.3. Consider the database instance $\mathcal{D}$ from Figure 1 and the example set E containing as positive and negative examples all tuples marked with “✓” and “✗”, respectively. Recall queries Q_1 and Q_2 from Example 3.2. Query Q_1 does not satisfy E over $\mathcal{D}$, because $\text{schema}(E) \neq \text{schema}(Q_1)$. Query Q_2 does not satisfy E over $\mathcal{D}$, because $Q_2(\mathcal{D})$ does not return the positive example

$$\{A : 1, B : 1, C : 2, D : 1, E : 5\}.$$

The query $\sigma_{A=1}(\sigma_{B \neq 3}(R_6))$ satisfies E over $\mathcal{D}$, as it returns all the positive examples (and none of the negative examples) when applied to the database $\mathcal{D}$.

Query Classes. In this paper, we are interested in learning queries from example sets. The difficulty of this problem can depend on the operators allowed within the query, as well as on the type of example sets allowed. We use $\mathcal{Q}^O$, where O is a list of operators from $\{\bowtie, \sigma(f), \pi\}$, to denote a class of RA queries. The query class $\mathcal{Q}^O$ contains all queries using only the operators in O , where $f \subseteq \Theta$ lists the permitted selection operators. We note that we always assume that $f \neq \emptyset$, if O contains selection. In our queries, the operators $\bowtie, \sigma, \pi$ (when appearing) are evaluated in this order (i.e., joins, then selections, and finally projection).

We may sometimes be interested in bounding the size of a query. Learning queries of small, bounded, size is of interest, as such queries will avoid over-fitting of the given example set. They will also be simpler to understand, and thus, can be of more interest to the user. To formalize this notion, we define the *size* of a query Q , denoted $\text{size}(Q)$, recursively, as:

1. 1, if $Q = R$, for some relation R .
2. $\text{size}(Q') + 1$, if $Q = \pi_{A_1, \dots, A_n}(Q')$.
3. $\text{size}(Q') + 1$, if $Q = \sigma_\psi(Q')$;
4. $\text{size}(Q') + \text{size}(Q'')$, if $Q = Q' \bowtie Q''$;

Now, if $\mathcal{Q}^O$ is a class of queries, we use $\mathcal{Q}_k^O$ to denote the subset of $\mathcal{Q}^O$ that contains only queries Q such that $\text{size}(Q) \leq k$. We use $\mathcal{Q}$ as an abstract notation for a query class, if O , and possibly k , are not of importance.

EXAMPLE 3.4. Recall the queries in Example 3.2. The query $Q_1 = \pi_{A,D}(R_1 \bowtie R_6)$ has size 3, as it joins 2 relations, and then performs a projection. Query $Q_2 = \sigma_{D=2}(R_6)$ has size 2.

Problems of Interest. In this paper we will be interested in the complexity of determining whether there exists a satisfying query for an example set, as well as in finding such a query, if one exists. Thus, our two problems of interest are defined as follows.

PROBLEM 1 (SATISFIABILITY). Given a query class $\mathcal{Q}$, the satisfiability problem, denoted $\text{SAT}(\mathcal{Q})$ is defined as follows: Given an example set E and a database $\mathcal{D}$, determine whether there exists a query $Q \in \mathcal{Q}$ that satisfies E over $\mathcal{D}$.

PROBLEM 2 (LEARNING). Given a query class $\mathcal{Q}$, the learning problem, denoted $\text{LEARN}(\mathcal{Q})$ is defined as follows: Given an example set E and a database $\mathcal{D}$, find a query $Q \in \mathcal{Q}$ that satisfies E over $\mathcal{D}$, or determine that such a query does not exist.

We will be interested in a bounded variant of the above two problems, defined as follows.

PROBLEM 3 (BOUNDED SATISFIABILITY). Given a query class $\mathcal{Q}$, the bounded-query-size satisfiability problem, denoted $\text{BSAT}(\mathcal{Q})$ is defined as follows: Given an example set E , a database $\mathcal{D}$, and a value k , determine whether there exists a query $Q \in \mathcal{Q}$ that satisfies E over $\mathcal{D}$ such that the size of Q is bounded by k .

The bounded-query-size learning problem, denoted $\text{BLEARN}(\mathcal{Q})$ is defined similarly.

The complexity of these problems depends highly upon the types of example sets and queries allowed. Thus, we will study these problems for a variety of settings. We note that even when we can determine that if a satisfying query exists, it must be one among a small predetermined set of possible queries $Q_1, \dots, Q_k$, these problems may still be hard, since it may not be possible to determine in polynomial time whether tuples in the example set are in the result of evaluating some query Q_i over a given database instance $\mathcal{D}$.

Constants and Parameters. The complexity of satisfiability and learning will obviously depend on the size of the input. In our problems, there are many different natural components to the problem, whose size may be of interest:

1. *The number of tuples in the database, denoted $|\mathcal{D}|$:* Finding a satisfying query often involves evaluating a query Q over the database, to check for satisfaction of the example set. The size of the database will impact the time it takes to evaluate a query. In addition, the search for a satisfying query is sometimes driven by looking for tuples in the database, that can be used to derive positive tuples in the example set. If there are more tuples in the database, there will naturally be more candidate tuples of origin, and hence, more candidate queries.
2. *The size of the query to be learned, for bounded-sized queries:* The complexity of learning queries from a class $\mathcal{Q}_k$ (containing queries of size at most k), is obviously dependent on k . For larger values of k , there will be more queries to consider, and query evaluation will be more expensive.
3. *The size of the database schema, denoted $|\text{schema}(\mathcal{D})|$:* A satisfying query may join relations from the database,

or perform selections over attributes in the database. The maximum number of such joins will be determined by the number of relations in the database. The number of different selections considered may also be a function of the number of attributes in the database. Hence the size of the schema can directly affect the size of a query to be learned.

4. *The number of examples in the example set, denoted $|E|$:* As mentioned above, we may search for a satisfying query by looking for tuples that can (resp., cannot) be used to derive positive (resp., negative) examples. Hence, when there are more examples, it may be more difficult to find a satisfying query.

Among the four components considered above, the first component, i.e., $|\mathcal{D}|$, is likely to be the largest by far, in practice. Hence, we will never assume that the number of tuples is bounded in size. For each of the other three components, it is of interest to study the complexity of satisfiability and learning, when they may be bounded in size. We differentiate between two types of bounds on k , $|\text{schema}(\mathcal{D})|$ and $|E|$: (1) the component is assumed to be a constant or (2) the component is a parameter to the problem. For the former, we would say that an algorithm running in time $\mathcal{O}(|\mathcal{D}|^c)$, where c is one of k , $|\text{schema}(\mathcal{D})|$ or $|E|$, is in PTIME. For the latter, we consider parameterized complexity analysis, and thus, would consider an algorithm running in time $\mathcal{O}(p(|\mathcal{D}|) \cdot f(c))$, where p is a polynomial function over $|\mathcal{D}|$ and f is a (computable) function over c , to be fixed parameter tractable. Note that when $|\mathcal{D}|$ is large, and c is not negligible, an algorithm with the latter runtime will be asymptotically faster.

In light of the above discussion, we consider four flavors of our problems. In Section 4, we study satisfiability and learning when there are no bounded components to our problem. Then, in Section 5, we assume that the queries to be learned are of bounded size, i.e., in $\mathcal{Q}_k$, and study the complexity of our problems both when k is a constant, and when it is a parameter. Section 6 considers the size of the schema, and again, analyzes the complexity of our problem both when the schema size is a constant, and when it is a parameter. Finally, in Section 7 we consider the complexity of our problems when the number of examples is a constant, or a parameter.

REMARK 3.1. *It is important to note that throughout our paper, we consider learning both queries that are unbounded in size, and those that are in $\mathcal{Q}_k$. In all sections, other than Section 5, we assume that k is part of the input of the problem. Only in Section 5 do we assume that k is a constant, or a parameter of the problem. Note that even when the query to be learned is bounded in size, it can be natural to view k as part of the input. This is in the spirit of the well-known measure of combined complexity.*

4. NO PARAMETERS

In this section, we consider the complexity of learning queries from examples, when there are no parameters, i.e., all components of the problem are part of the input. In particular, we do not assume a constant bound on the size of the schema or example set. In addition, even when learning bounded-sized queries from a class $\mathcal{Q}$ (queries of size at most k), we do not consider k to be a constant in our

complexity analysis (and thus, e.g., a runtime of $|\mathcal{D}|^k$ is not polynomial). This analysis is in the spirit of the standard notion of combined complexity.

We start by considering the class of queries $\mathcal{Q}^\pi$, i.e., queries containing only the project operator. Recall that example sets contain tuples, i.e., sets of the form $\{A_1 : c_1, \dots, A_n : c_n\}$. Hence, the queries in $\mathcal{Q}^\pi$ that can satisfy a given example set E are of the form $\pi_{A_1, \dots, A_n} R$, where R is a relation.² Clearly, given a relation R , one can determine in polynomial time whether $\pi_{A_1, \dots, A_n} R$ satisfies E . Hence, in the remainder of the paper, we do not consider learning queries that contain only the project operation.

The following theorem summarizes the complexity of learning queries of unbounded size, for different combinations of operators. For tractable cases, we show how learning can be performed efficiently, while for intractable cases, we show that determining satisfiability is already hard.

THEOREM 4.1. *Let f be any nonempty subset of Θ .*

1. $\text{LEARN}(\mathcal{Q}^{\sigma(f)})$, $\text{LEARN}(\mathcal{Q}^{\bowtie})$ and $\text{LEARN}(\mathcal{Q}^{\sigma(f), \bowtie})$ are in PTIME;
2. $\text{SAT}(\mathcal{Q}^{\pi, \sigma(f)})$ is NP-complete;
3. $\text{SAT}(\mathcal{Q}^{\pi, \bowtie})$ and $\text{SAT}(\mathcal{Q}^{\pi, \sigma(f), \bowtie})$ are both NP-hard and coNP-hard, and in DP.

We explain the intuition behind these results, while formal proofs are omitted, due to lack of space. Tractability of $\text{LEARN}(\mathcal{Q}^{\sigma(f)})$ follows from the fact that queries in $\mathcal{Q}^{\sigma(f)}$ can be evaluated in polynomial time, and a single most specific query, for a given set of examples, can be identified. This is demonstrated next.

EXAMPLE 4.2. *Consider the relation R_6 from Figure 1, as well as the example set containing as positive and negative examples, the tuples marked by “✓” and “✗”, respectively. When learning a query in $\mathcal{Q}^{\sigma(f)}$, we can simply consider the single query containing the most restrictive conditions allowed by f , given the positive examples. For example, if $f = \Theta_{\text{pos}}$, this is the query³*

$$\sigma_{A=1}(\sigma_{1 \leq B \leq 5}(\sigma_{2 \leq C \leq 4}(\sigma_{1 \leq D \leq 4}(\sigma_{3 \leq E \leq 5}(R_6))))),$$

which does not satisfy the example set. Hence, no satisfying query exists in $\mathcal{Q}^{\sigma(\Theta_{\text{pos}})}$. When allowing $\neq$ -comparisons, additional selections can be added, such as $\sigma_{B \neq 3}$, $\sigma_{D \neq 3}$ and $\sigma_{E \neq 4}$. Such selections will yield a satisfying query. As will be shown later, when the class of queries considered is bounded in size, satisfiability becomes intractable, as, intuitively, it is difficult to choose a small subset of selections that yield a satisfying query.

Interestingly, $\text{LEARN}(\mathcal{Q}^{\bowtie})$ (Claim 1 of Theorem 4.1) is tractable, even though fully evaluating queries for this class of queries is intractable, as the query may return an exponential number of tuples. Intuitively, this follows from two observations. First, given a specific tuple t and a database

²Technically, one may also consider queries of the form $\pi_{A_1, \dots, A_n}(\pi_{B_1, \dots, B_m} R)$, where each A_i is among B_j . However, such queries may be safely ignored, as it only increases query size, but does not increase expressive power. Note also that the order in which attributes are listed in a projection operator is of no importance.

³We use $\sigma_{c_1 \leq A \leq c_2}$ as a shorthand for $\sigma_{A \geq c_1} \sigma_{A \leq c_2}$.

	A_{x_1}	$A_{\bar{x}_1}$	A_{x_2}	$A_{\bar{x}_2}$	A_{x_3}	$A_{\bar{x}_3}$	A_1	A_2
$t_{x_1}^p$	1	0	0	0	0	0	x_1	0
$t_{x_1}^n$	0	1	0	0	0	0	x_1	0
$t_{x_1}^b$	1	1	0	0	0	0	0	1
$t_{x_2}^p$	0	0	1	0	0	0	x_2	0
$t_{x_2}^n$	0	0	0	1	0	0	x_2	0
$t_{x_2}^b$	0	0	1	1	0	0	0	1
$t_{x_3}^p$	0	0	0	0	1	0	x_3	0
$t_{x_3}^n$	0	0	0	0	0	1	x_3	0
$t_{x_3}^b$	0	0	0	0	1	1	0	1
$t_{\phi_1}^1$	0	1	0	0	0	0	ϕ_1	1
$t_{\phi_1}^2$	0	0	1	0	0	0	ϕ_1	1
$t_{\phi_1}^3$	0	0	0	0	0	1	ϕ_1	1
$t_{\phi_2}^1$	1	0	0	0	0	0	ϕ_2	1
$t_{\phi_2}^2$	0	0	1	0	0	0	ϕ_2	1
$t_{\phi_2}^3$	0	0	0	0	1	0	ϕ_2	1
t_0	0	0	0	0	0	0	0	0

Figure 2: Reduction from 3-SAT for Example 4.4.

$\mathcal{D}$, it is possible to determine in polynomial time whether t is returned by a query $Q \in \mathcal{Q}^{\bowtie}$, while avoiding full evaluation of $Q(\mathcal{D})$ (which may be exponential in size). Second, the result of a join query monotonically decreases when additional joins are added (as long as the new joins do not introduce new attributes). Thus, we can create a most specific query returning the positive examples, and then check (without full evaluation) whether the negative examples are returned. Tractability of $\text{LEARN}(\mathcal{Q}^{\sigma(f), \bowtie})$ follows by combining the ideas for $\mathcal{Q}^{\bowtie}$ and for $\mathcal{Q}^{\sigma(f)}$.

EXAMPLE 4.3. Consider the relations $R_1, \dots, R_6$ from the database $\mathcal{D}$ appearing in Figure 1, and the example set E_1 containing a single positive example

$$t_p = \{A:1, B:1, C:1, D:1\}$$

and a single negative example

$$t_n = \{A:1, B:1, C:2, D:1\}.$$

In order to learn a query in $\mathcal{Q}^{\bowtie}$ satisfying E_1 , we first observe that R_6 cannot participate in a satisfying query, as it has an attribute not appearing in E_1 . Next, we note that all remaining relations, other than R_5 , can potentially be in a join query producing t_p . This follows, since for all R_i where $i < 5$, it holds that $\pi_{\text{schema}(R_i)}(\{t_p\}) \in R_i$. Hence, when there is no bound on the query size, there is a satisfying query if and only if $Q = R_1 \bowtie R_2 \bowtie R_3 \bowtie R_4$ is such a satisfying query. Since, $\pi_{\text{schema}(R_1)}(\{t_n\}) \notin R_1$, we can immediately conclude that Q is indeed satisfying (as it produces the positive example, but not the negative one).

Intractability of $\text{SAT}(\mathcal{Q}^{\pi, \sigma(f)})$, i.e., Claim 2 of Theorem 4.1) can be shown by a reduction from 3-SAT, as demonstrated in the following example.

EXAMPLE 4.4. Consider a 3-SAT formula, $\Phi = \phi_1 \wedge \phi_2$, where

$$\phi_1 = x_1 \vee \neg x_2 \vee x_3 \quad \phi_2 = \neg x_1 \vee \neg x_2 \vee \neg x_3.$$

We create the relation R as in Figure 2. The relation R has columns A_{x_i} and $A_{\bar{x}_i}$, for every variable x_i , as well as

columns A_1 and A_2 . Observe that R has three tuples $t_{x_i}^p$, $t_{x_i}^n$ and $t_{x_i}^b$, for each variable x_i . In $t_{x_i}^p$ (resp. $t_{x_i}^n$) attribute A_1 has the value x_i (i.e., the name of the variable x_i) and attribute A_{x_i} (resp $A_{\bar{x}_i}$) has the value 1. In $t_{x_i}^b$, A_{x_i} , $A_{\bar{x}_i}$ and A_2 have the value 1. In addition, R has three tuples $t_{\phi_j}^1$, $t_{\phi_j}^2$, $t_{\phi_j}^3$ for each ϕ_j , with value ϕ_j (the name of the j -th clause) in attribute A_1 . In addition, $t_{\phi_j}^k$ has the value 1 in the attribute corresponding to the negation of the k -th variable in ϕ_j . Finally, R also has a tuple containing only the value 0.

We create an example set, with a positive example for each variable, and for each clause, and a single negative example, as follows.

	A_1	A_2
✓	ϕ_1	1
✓	ϕ_2	1
✓	x_1	0
✓	x_2	0
✓	x_3	0
✓	0	0
✗	0	1

To simplify, consider only queries in $\mathcal{Q}^{\pi, \sigma(\{=\})}$. Clearly, a satisfying query must require exactly one of A_{x_i} or $A_{\bar{x}_i}$ to have the value 0. Otherwise, (1) if neither are required, the tuple $t_{x_i}^b$ will derive the negative example and (2) if both are required, we will not derive the positive example $\{A_1 : x_i, A_2 : 0\}$. Thus, a satisfying query Q corresponds to a truth assignment, in which we assign x_i true if and only if Q contains the selection $A_{x_i} = 0$. In addition, a satisfying query Q will derive each positive example $\{A_1 : \phi_j, A_2 : 1\}$. This implies satisfaction of ϕ_j by the truth assignment. Thus, for example, the satisfying truth assignment $x_1 = \top, x_2 = \perp, x_3 = \perp$ corresponds to the satisfying query

$$\pi_{A_1, A_2}(\sigma_{A_{x_1}=0}(\sigma_{A_{\bar{x}_2}=0}(\sigma_{A_{\bar{x}_3}=0}(R)))).$$

Finally, when both join and projection operators are permitted, as in Claim 3 of Theorem 4.1, a new difficulty arises. Not only can it be difficult to find a satisfying query, but given a specific query Q , it can be intractable to determine whether Q satisfies an example set. This contrasts with all previously considered cases, in which it is possible to efficiently decide satisfiability of a given query. For both $\text{SAT}(\mathcal{Q}^{\pi, \bowtie})$ and $\text{SAT}(\mathcal{Q}^{\pi, \sigma(f), \bowtie})$, NP-hardness and coNP-hardness follow from the fact that we must both determine that all positive answers are returned, and that no negative answers are returned. Determining whether a tuple is in a query result is NP-hard for these classes, while determining query emptiness is coNP-hard.

We now consider the complexity of learning queries of bounded size.

THEOREM 4.5. Let f be any nonempty subset of Θ .

1. $\text{BSAT}(\mathcal{Q}^{\bowtie})$, $\text{BSAT}(\mathcal{Q}^{\sigma(f)})$, $\text{BSAT}(\mathcal{Q}^{\sigma(f), \bowtie})$, and $\text{BSAT}(\mathcal{Q}^{\pi, \sigma(f)})$ are NP-complete;
2. $\text{BSAT}(\mathcal{Q}^{\pi, \bowtie})$ and $\text{BSAT}(\mathcal{Q}^{\pi, \sigma(f), \bowtie})$ are both NP-hard and coNP-hard, and in DP.

Bounding the size of the queries to be learned increases the complexity of satisfiability for query classes $\mathcal{Q}^{\bowtie}$, $\mathcal{Q}^{\sigma(f)}$ and $\mathcal{Q}^{\sigma(f), \bowtie}$. Satisfiability can be shown to be NP-complete, by

different reductions from the set cover problem, where query size corresponds to cover size. For the remaining classes, determining satisfiability is the same as in the unbounded cases, i.e., as in Theorem 4.1.

We demonstrate the intuition behind NP-completeness of $\text{BSAT}(\mathcal{Q}^{\bowtie})$ in the following example.

EXAMPLE 4.6. Consider a set $U = \{1, 2, 3, 4, 5\}$, and a collection of subsets $S = \{s_1, s_2, s_3, s_4\}$, where

$$\begin{aligned} s_1 &= \{1, 2, 3\} & s_2 &= \{2, 4\} \\ s_3 &= \{3, 4\} & s_4 &= \{4, 5\}. \end{aligned}$$

Our goal is to find a subset C of S with at most k sets, such that the union of the sets in C is precisely U .

Consider the database $\mathcal{D}_1$, containing the following four relations:

$$\begin{aligned} R_{s_1} &= \{\{A : 4\}, \{A : 5\}\} \\ R_{s_2} &= \{\{A : 1\}, \{A : 3\}, \{A : 5\}\} \\ R_{s_3} &= \{\{A : 1\}, \{A : 2\}, \{A : 5\}\} \\ R_{s_4} &= \{\{A : 1\}, \{A : 2\}, \{A : 3\}\}. \end{aligned}$$

Observe that R_{s_i} contains a tuple for each element that is not in s_i . Let E_1 be the example set containing no positive examples, and a negative example $\{A : v\}$, for every $v \in U$. Now, it is not difficult to see that there is a satisfying query in $\mathcal{Q}_k^{\bowtie}$ if and only if there is a cover of size at most k , for U . For example, the satisfying query $R_{s_1} \bowtie R_{s_4}$ corresponds to the set cover s_1, s_4 .

The intuition behind NP-completeness of $\text{BSAT}(\mathcal{Q}^{\sigma(f)})$ is demonstrated in the following example.

EXAMPLE 4.7. Consider again the set U , and the set of subsets S , from Example 4.6. Consider the database $\mathcal{D}_2$, containing the following single relation R , defined as:

		A_{s_1}	A_{s_2}	A_{s_3}	A_{s_4}
$\times$	t_1	0	1	1	1
$\times$	t_2	0	0	1	1
$\times$	t_3	0	1	0	1
$\times$	t_4	1	0	0	0
$\times$	t_5	1	1	1	0
$\checkmark$	t^1	1	1	1	1

where “ $\times$ ” and “ $\checkmark$ ” are used to mark negative and positive tuples, respectively, in the example set. (Note that every tuple in R is either a negative or positive example.)

Observe that R contains an attribute A_{s_i} for each subset s_i . There is a tuple t_v corresponding to each element $v \in U$, with 0 (resp. 1) for A_{s_i} if $v \in s_i$ (resp. $v \notin s_i$), as well as a tuple t^1 containing 1 in all attributes. Now, by careful observation one can see that there is a satisfying query in $\mathcal{Q}_{k+1}^{\sigma(f)}$ if and only if there is a cover of size at most k . For example, the satisfying query $\sigma_{A_{s_1}=1}(\sigma_{A_{s_4}=1}(R))$ corresponds to the set cover s_1, s_4 .

It is useful to note that both of these polynomial reductions are also fpt-reductions, with respect to parameter k . This will be used to show $W[2]$ -hardness, in the next section, when k is parameterized.

5. BOUNDING THE QUERY SIZE

In this section, we consider the complexity of learning bounded-sized queries from a class $\mathcal{Q}$ from examples, where we consider the size of permitted queries, i.e., k , to be constant or a parameter.

When k is constant, there are always a polynomial number of possible queries to be considered. Moreover, the evaluation of constant sized join queries is also polynomial. Hence, we derive the following result.

THEOREM 5.1. Let f be any nonempty subset of Θ and O be any subset of $\{\pi, \sigma(f), \bowtie\}$. Then $\text{BLEARN}(\mathcal{Q}^O)$ is in PTIME when the query size bound is constant.

Since Theorem 4.5 implies that satisfiability is hard when k is considered input, and Theorem 5.1 shows that learning is tractable when k is constant, it is of interest to determine the parameterized complexity of these problems with respect to k .

THEOREM 5.2. Let f be any nonempty subset of Θ . When the query size bound is considered a parameter

1. $\text{BSAT}(\mathcal{Q}^{\bowtie})$, $\text{BSAT}(\mathcal{Q}^{\sigma(f)})$, and $\text{BSAT}(\mathcal{Q}^{\sigma(f), \bowtie})$ are $W[2]$ -complete;
2. $\text{BSAT}(\mathcal{Q}^{\pi, \sigma(f)})$ is $W[3]$ -complete;
3. $\text{BSAT}(\mathcal{Q}^{\pi, \bowtie})$ is both $W[2]$ -hard and $\text{coW}[1]$ -hard;
4. $\text{BSAT}(\mathcal{Q}^{\pi, \sigma(f), \bowtie})$ is both $W[3]$ -hard and $\text{coW}[1]$ -hard.

For Claim 1, (queries without projection) $W[2]$ -hardness can be shown using the reductions from set cover used in Theorem 4.5, which were also fpt-reductions. These reductions were demonstrated in Examples 4.6 and 4.7. Membership in $W[2]$ is shown by reductions from each of $\text{BSAT}(\mathcal{Q}^{\bowtie})$, $\text{BSAT}(\mathcal{Q}^{\sigma(f)})$, and $\text{BSAT}(\mathcal{Q}^{\sigma(f), \bowtie})$ to set cover. The idea behind one such reduction, that of $\text{BSAT}(\mathcal{Q}^{\bowtie})$, is demonstrated next.

EXAMPLE 5.3. Consider the database $\mathcal{D}$ from Figure 1, and the example set E containing no positive examples, and three negative examples

$$\begin{aligned} t_n^1 &= \{A:1, B:1, C:2, D:1\} & t_n^2 &= \{A:1, B:2, C:1, D:2\} \\ t_n^3 &= \{A:2, B:2, C:2, D:2\}. \end{aligned}$$

We define an instance of the set cover problem over $\{t_n^1, t_n^2, t_n^3, A, B, C, D\}$, with sets

$$\begin{aligned} s_1 &= \{t_n^1, t_n^2, t_n^3, A, C, D\} & s_2 &= \{t_n^1, t_n^2, B, C, D\} \\ s_3 &= \{t_n^3, A, B, D\} & s_4 &= \{t_n^1, t_n^3, A, C, D\} \\ s_5 &= \{t_n^3, A, B, C\} \end{aligned}$$

Intuitively, each set s_i contains the attributes of R_i , as well as all negative examples disqualified from the result by including R_i . Then, a cover of size k of $\{t_n^1, t_n^2, t_n^3, A, B, C, D\}$ can be shown to correspond to a satisfying join query of size k , e.g., the cover s_2, s_3 corresponds to the satisfying query $R_2 \bowtie R_3$. We note that in this example E contains only negative tuples. In our proof we also consider the case that E contains positive examples.

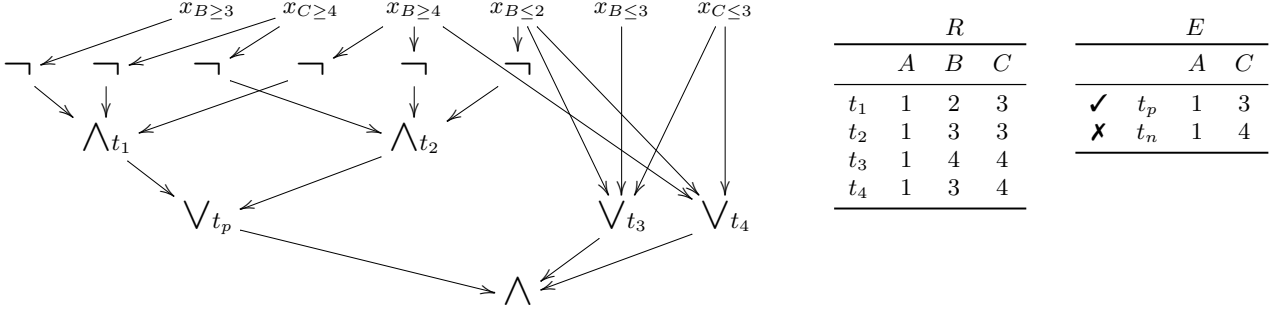


Figure 3: Circuit of weft 3.

Showing Claim 2 of Theorem 5.2, i.e., $W[3]$ -completeness of $\text{BSAT}(\mathcal{Q}^{\pi, \sigma(f)})$, is more difficult as few natural complete problems for $W[3]$ are known. To show membership in $W[3]$, we give a reduction to the *weighted circuit satisfiability problem* with weft 3, a canonical complete problem for $W[3]$ (see [13]). To show $W[3]$ -hardness, we give a reduction from the *weighted circuit anti-monotone normalized satisfiability problem* with weft 3, which is also complete for $W[3]$. The full details are deferred to the appendix, but in the following example, we demonstrate the idea behind our reduction showing membership in $W[3]$.

EXAMPLE 5.4. We briefly review the *weighted circuit satisfiability problem*. In the *weighted circuit satisfiability problem*, we are given a Boolean circuit, with a single output gate, which is an AND gate. The circuit may contain both small gates of fan-in at most 2, and large AND/OR gates with arbitrary fan-in. The *weft* of a circuit is the maximum number of large gates on any path from an input to the output gate. Given a Boolean circuit, and a number k , the *weighted circuit satisfiability problem* is to determine whether there exists an input of Hamming weight at most k that is accepted by the circuit.⁴ For circuits of weft t , this problem is $W[t]$ -complete, given parameter k .

Consider the database $\mathcal{D}$ containing the relation R from Figure 3, as well as the example set E depicted in this figure. To simplify, in this example, we assume that f is the set $\{\leq, \geq\}$, i.e., we reduce bounded satisfiability of $\mathcal{Q}^{\pi, \sigma(\{\leq, \geq\})}$ to the weighted circuit satisfiability problem. We construct a circuit of weft 3, as depicted in the same figure.

The input variables correspond to selections on the attributes. Now, observe that the two possible origins for t_p in the result of a query are t_1 and t_2 . Hence, at least one of these two tuples must remain after the selections. Therefore, all variables corresponding to selections removing one of these tuples are negated and fed into an AND gate, and then these two AND gates are the input of an OR gate. This ensures that a satisfying assignment of the input will leave at least one of t_1 and t_2 . Observe that there are two possible origins for t_n , namely t_3 and t_4 . Both of these must be removed by the selections of the query. This is ensured by using OR gates that combine all selections removing the tuples. Finally, it is not difficult to see that an assignment of weight k corresponds to a query with k selections, and one projection (and one relation), i.e., to a query of size $k + 2$.

⁴The classic weighted circuit satisfiability problem is to determine whether there exists an input of Hamming weight precisely k . The flavor of the problem considered here is also $W[3]$ -complete.

The problem of $\text{BSAT}(\mathcal{Q}^{\pi, \boxtimes})$, i.e., Claim 3 of Theorem 5.2 is both $W[2]$ -hard and $\text{co}W[1]$ -hard. Observe that $W[2]$ -hardness follows immediately from Claim 1, which shows that $\text{BSAT}(\mathcal{Q}^{\boxtimes})$ is $W[2]$ -complete. The $\text{co}W[1]$ -hardness comes from the difficulty of determining non-satisfiability of negative examples, even if the target query is known. Similarly, in Claim 4, i.e., $\text{BSAT}(\mathcal{Q}^{\pi, \sigma(f), \boxtimes})$, we have $\text{co}W[1]$ -hardness for the same reason, and $W[3]$ -hardness immediately follows from Claim 2.

6. BOUNDING THE SCHEMA SIZE

In this section, we consider a bound on the size of the schema of the database. Thus, the number of tuples in the database is unbounded, but we assume that the number of relations, and the number of attributes per relation, is bounded. This is a natural assumption, as the schema will typically be significantly smaller than the actual data in a database.

For some query classes (in particular, those without “ $\neq$ ”-selections), bounding the schema size also bounds the size of a query to be learned. For these classes, the complexity can only decrease in comparison to that in the previous section. In other cases, bounding the schema size does not imply a bound on the size of a query to be learned, and hence, new complexity bounds must be formulated. Note that, as in Section 4, even when learning bounded-sized queries from a class $\mathcal{Q}$ (queries of size at most k), we do not consider k to be a constant in our complexity analysis, but rather only consider the schema size to be a constant (or a parameter). Finally, we will not parameterize an input of a problem if the non-parameterized version of the problem is tractable.

We begin by considering queries that may contain any of the operators natural join, projection and *positive* selection, i.e., queries that may contain “ $=$ ”, “ $\leq$ ” and “ $\geq$ ”, but do not contain “ $\neq$ ”. In this case, bounding the schema size effectually bounds the size of a query, as:

- the number of joins will be at most the size of the schema and
- for each attribute, there are at most three comparisons of interest (one each using the operators “ $=$ ”, “ $\leq$ ” and “ $\geq$ ”).

Therefore, the following theorem follows immediately from Theorem 5.1, which considers queries of size bounded by a constant.

THEOREM 6.1. *Let f_{pos} be any nonempty subset of Θ_{pos} and O be any subset of $\{\pi, \sigma(f_{\text{pos}}), \boxtimes\}$. Then, $\text{LEARN}(\mathcal{Q}^O)$*

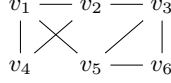


Figure 4: A graph of maximum degree 3.

and $\text{BLEARN}(\mathcal{Q}^O)$ is in PTIME, if the schema is of constant size.

Theorem 6.1 does not imply fixed parameter tractability for queries with positive selection. In particular, as shown in Section 5, determining satisfiability, for queries of bounded size, is not fixed parameter tractable, regardless of the operators allowed in the queries. Interestingly, bounding the schema size can sometimes lead to fixed parameter tractability, as shown in the following theorem.

THEOREM 6.2. *Let f_{pos} be any nonempty subset of Θ_{pos} . When the schema size is a parameter,*

1. $\text{BLEARN}(\mathcal{Q}^{\bowtie}), \text{BLEARN}(\mathcal{Q}^{\sigma(f_{\text{pos}})}), \text{BLEARN}(\mathcal{Q}^{\sigma(f_{\text{pos}}), \bowtie}), \text{LEARN}(\mathcal{Q}^{\pi, \sigma(\{=\})})$ and $\text{BLEARN}(\mathcal{Q}^{\pi, \sigma(\{=\})})$ are FPT;
2. $\text{SAT}(\mathcal{Q}^{\pi, \sigma(\{\leq, \geq\})})$ and $\text{BSAT}(\mathcal{Q}^{\pi, \sigma(\{\leq, \geq\})})$ are in $W[3]$;
3. $\text{SAT}(\mathcal{Q}^{\pi, \bowtie}), \text{BSAT}(\mathcal{Q}^{\pi, \bowtie}), \text{SAT}(\mathcal{Q}^{\pi, \sigma(f_{\text{pos}}), \bowtie}),$ and $\text{BSAT}(\mathcal{Q}^{\pi, \sigma(f_{\text{pos}}), \bowtie})$ are $W[1]$ -hard and co- $W[1]$ -hard.

As Theorem 6.2 shows, in the absence of projection, or when projection is combined with “=”-selections, learning is fixed parameter tractable. Thus, Claim 1 is shown by a counting argument on the number of queries that must be considered. For Claim 2, observe that considering only “ $\leq$ ” and “ $\geq$ ”-selections effectually bounds the size of a query that must be considered, as there are at most two selections of interest per attribute. Hence, this result follows from Claim 2 of Theorem 5.2. Finally, Claim 3 can be shown to follow from the fact that it is $W[1]$ -hard to determine whether a given tuple is in the result of a query, in these classes.

Finally, in the following theorem, we consider queries that can have “ $\neq$ ”-selections. Even when the schema is of constant size, satisfiability is NP-complete, for all classes that are not tractable in the absence of parameters.

THEOREM 6.3. *Let O be any subset of $\{\pi, \bowtie\}$ and let $O_{\text{neg}} = O \cup \{\sigma(\{\neq\})\}$. Suppose that the schema is of constant size.*

1. If $\pi \in O_{\text{neg}}$ then $\text{SAT}(\mathcal{Q}^{O_{\text{neg}}})$ is NP-complete.
2. $\text{BSAT}(\mathcal{Q}^{O_{\text{neg}}})$ is NP-complete.

For both claims, membership in NP is easily shown, since queries can be evaluated in polynomial time, when the size of the join is bound by a constant. Hardness is shown by a reduction from 3-SAT in Claim 1, and by a reduction from the degree-3 vertex cover problem in Claim 2. We demonstrate the latter reduction in the following example.

EXAMPLE 6.4. *Consider the undirected and connected graph G in Figure 4, in which each vertex has degree at most three. It is well known that such graphs have a chromatic number of no more than three (if they contain at least five vertices), and*

that a 3-coloring can be found in linear time. For example, the following is a 3-coloring of G :

$$\text{Red} = \{v_4, v_6\} \quad \text{Blue} = \{v_1, v_3\} \quad \text{Green} = \{v_2, v_5\}$$

Our goal is to find a vertex cover of size at most k , for such a degree-3 graph. This problem is NP-complete [15].

We create a relation R containing the tuples:

$$\begin{array}{ll} \{r : 0, b : v_1, g : v_2\} & \{r : v_4, b : v_1, g : 0\} \\ \{r : 0, b : v_1, g : v_5\} & \{r : 0, b : v_3, g : v_2\} \\ \{r : v_4, b : 0, g : v_2\} & \{r : 0, b : v_3, g : v_5\} \\ \{r : v_6, b : v_3, g : 0\} & \{r : v_6, b : 0, g : v_5\} \\ \{r : 0, b : 0, g : 0\} & \end{array}$$

Observe that each edge in the graph corresponds with a tuple in the relation, with the vertices appearing in the attribute corresponding to their color. In addition, observe that R also has a tuple containing only the value 0. The example set E contains all tuples associated with edges as negative examples, and the tuple with only 0-s as a positive example.

Now, it is not difficult to see that a vertex cover U corresponds to a satisfying query that has an inequality selection for each vertex in U . For example, the vertex cover $\{v_1, v_2, v_3, v_5\}$ corresponds to the satisfying query

$$\sigma_{b \neq v_1}(\sigma_{g \neq v_2}(\sigma_{b \neq v_3}(\sigma_{g \neq v_5}(R)))) .$$

Given the NP-completeness result of Theorem 6.3, it is interesting to consider the complexity of learning queries with “ $\neq$ ”-selections, when both the schema and query are bounded in size. Note that if the query is of bounded size, it is intractable to determine satisfiability of queries with selections, even if the query size is a parameter, as shown in Theorem 5.2. Interestingly, the following theorem shows that by considering both the schema size and the query size as parameters, we derive fixed parameter tractability.

THEOREM 6.5. *When both the size of the schema and the size of the query are parameters, $\text{BLEARN}(\mathcal{Q}^{\sigma(\{\neq\})}), \text{BLEARN}(\mathcal{Q}^{\pi, \sigma(\{\neq\})})$ and $\text{BLEARN}(\mathcal{Q}^{\sigma(\{\neq\}), \bowtie})$ are in FPT.*

The cases of $\mathcal{Q}^{\sigma(\{\neq\})}$ and $\mathcal{Q}^{\pi, \sigma(\{\neq\})}$ follow from the following counting argument. Let $e_1, \dots, e_n$ be the negative examples. For a given

$$e_i = \{A_1 : c_1, \dots, A_s : c_s\},$$

there are s inequality conditions that can be used to disqualify e_i from the result of a query, namely $A_i \neq c_i$, for all $i \leq s$. Therefore, we can find a satisfying query, if one exists, by repeatedly choosing a negative example, considering each disqualifying selection, and then filtering out any other negative examples that are also disqualified by this process. As we must find a query of size at most k , this process will be continued up to a depth of k , thereby creating s^k queries to consider. Since s is bound by the schema size, which is a parameter, and k is also a parameter, this yields fixed parameter tractability.

The above argument can be adapted to the case of $\mathcal{Q}^{\sigma(\{\neq\}), \bowtie}$. Suppose that the schema is of size s . Then, there are at most 2^s join combinations that must be considered. For each such query, we must disqualify any remaining negative examples, as before. Again, given a negative example with at most s attributes, there are at most s inequality conditions that can be used to disqualify the tuple. Therefore, for each join

query, we repeatedly choose a remaining negative example, and a disqualifying condition, and then filter out any other negative examples that are disqualified by this process. We continue this process up to a depth of k , yielding at most $2^s s^k$ queries to be considered. Since both s and k are parameters, this yields our desired FPT result.

7. BOUNDING NUMBER OF EXAMPLES

In this section we consider the complexity of satisfiability and learning queries, when the number of examples is considered to be constant or a parameter. Since the examples are provided by the users, the example set is likely to be significantly smaller than the size of the database.

We first consider query classes which include the join operator. The following theorem summarizes the complexity of satisfiability for all classes of queries with join, that are intractable when no parameters are given. Indeed, even when the number of examples is a constant, the complexity remains the same as when no parameters are available, i.e., as in Theorems 4.1 and 4.5. However, a new proof is required for Claim 1 of Theorem 7.1, as the proof of Theorem 4.5 used an unbounded number of examples to show NP-completeness of $\text{BSAT}(Q^{\bowtie})$.

THEOREM 7.1. *Let f be any nonempty subset of Θ . When the number of examples is a constant,*

1. $\text{BSAT}(Q^{\bowtie})$ and $\text{BSAT}(Q^{\sigma(f), \bowtie})$ are NP-complete;
2. $\text{SAT}(Q^{\pi, \bowtie})$, $\text{SAT}(Q^{\pi, \sigma(f), \bowtie})$, $\text{BSAT}(Q^{\pi, \bowtie})$, and $\text{BSAT}(Q^{\pi, \sigma(f), \bowtie})$ are NP-hard, coNP-hard, and in DP.

The following example demonstrates the reduction from set cover used to show that $\text{BSAT}(Q^{\bowtie})$ is NP-hard.

EXAMPLE 7.2. *Consider again the set $U = \{1, 2, 3, 4, 5\}$ and collection of subsets $S = \{s_1, s_2, s_3, s_4\}$, where*

$$\begin{array}{ll} s_1 = \{1, 2, 3\} & s_2 = \{2, 4\} \\ s_3 = \{3, 4\} & s_4 = \{4, 5\} \end{array}$$

from Example 4.6. We create a database with relations R_1, R_2, R_3, R_4 , with the following tuples

$$\begin{array}{ll} R_1 = \{A_1 : 1, A_2 : 1, A_3 : 1\} & R_2 = \{A_2 : 1, A_4 : 1\} \\ R_3 = \{A_3 : 1, A_4 : 1\} & R_4 = \{A_4 : 1, A_5 : 1\} \end{array}$$

Note that the attributes in relation R_i correspond to the elements in set s_i . We consider the example set containing a single positive example

$$\{A_1 : 1, A_2 : 1, A_3 : 1, A_4 : 1, A_5 : 1\}.$$

Now, it is easy to see that there is a satisfying query in $Q_k^{\bowtie}$ if and only if there is a cover of size at most k , for U . For example, the satisfying query $R_1 \bowtie R_4$ corresponds to the set cover s_1, s_4 .

We now consider the complexity of the learning and satisfiability problems for query classes which allow selection, but do not allow join. Recall that in the absence of projection, for unbounded queries, i.e., $Q^{\sigma(f)}$, learning is in PTIME, even if there are no parameters given. Hence, we consider only the bounded satisfiability problem for the query class $Q^{\sigma(f)}$, and both the bounded and unbounded variants for the class $Q^{\pi, \sigma(f)}$.

THEOREM 7.3. *Let f be any nonempty subset of Θ . Let e be the number of examples in the example set.*

1. $\text{BLEARN}(Q^{\sigma(f)})$ is in FPT when e is a parameter;
2. $\text{SAT}(Q^{\pi, \sigma(f)})$ is W[3]-complete when e is a parameter;
3. $\text{BSAT}(Q^{\pi, \sigma(f)})$ is NP-complete even when e is constant.

Let k denote the query size bound in Claims 1 and 3. To show Claim 1, observe that at most one selection is required to remove each negative example. Thus, if a satisfying query exists, then there is one of size at most $e+1$. If $k \geq e+1$, then FPT complexity follows from the fact that $\text{LEARN}(Q^{\sigma(f)})$ is PTIME, in the absence of parameters (Claim 1 of Theorem 4.1). In this case, it is sufficient to learn any satisfying query (in PTIME), and then to remove redundant selections, until a query of size at most k is derived. If $k \leq e$, more care must be taken to prove the FPT complexity result. The proof involves dividing the negative examples into at most $k-1$ sets, where each set must be disqualified by a single selection, and then applying a similar process to find a satisfying query. We note that this yields FPT time, since e is a parameter, and the number of ways to divide e into sets is a function of e , and not of the database size.

We next consider Claim 2 of Theorem 7.3, i.e., W[3]-completeness of $\text{SAT}(Q^{\pi, \sigma(f)})$, when the number of examples is a parameter. Our proof of Claim 2 of Theorem 5.2 cannot be applied to show W[3]-completeness in the current case, as our problem considers queries of unbounded size, while Theorem 5.2 considered queries of bounded size, i.e., $\text{BSAT}(Q^{\pi, \sigma(f)})$. To show membership in W[3], we give a reduction to the *weighted circuit satisfiability problem* with weft 3 and to show W[3]-hardness, we give a reduction from the *weighted circuit anti-monotone normalized satisfiability problem* with weft 3, as in Claim 2 of Theorem 5.2.

In the following example, we demonstrate the reduction for the W[3]-hardness result.

EXAMPLE 7.4. *In the weighted circuit anti-monotone normalized satisfiability problem with weft 3, all inputs are negated, and all paths from inputs to the root of the circuit tree pass through large AND gates, large OR gates, and a final large AND gate. Large gates can have arbitrary fan-in. The problem is to find a satisfying assignment of precisely Hamming weight k .*

For example, the following formula Ψ can be expressed using an anti-monotone circuit of weft 3. (In the interest of simplifying the example, we give small formulas, which could also be expressed using small gates. A larger formula would require the use of large gates.)

$$\begin{array}{ll} \phi_1 = \neg v_1 \wedge \neg v_2 & \phi_2 = \neg v_3 \wedge \neg v_1 \\ \phi_3 = \neg v_3 \wedge \neg v_4 & \phi_4 = \neg v_1 \wedge \neg v_3 \\ \Phi_1 = \phi_1 \vee \phi_2 & \Phi_2 = \phi_3 \vee \phi_4 \\ \Psi = \Phi_1 \wedge \Phi_2 & \end{array}$$

The problem at hand is to find a satisfying assignment of weight precisely k . To simplify, in this example, we assume that $k = 2$.

We create a database with a single relation R , as in the left-hand side of Figure 5. Observe that R has an attribute A_{ϕ_i} for each formula ϕ_i and k attributes for each variable v_j , as well as two additional attributes B and C . The relation R has tuples, as follows:

	A_{ϕ_1}	A_{ϕ_2}	A_{ϕ_3}	A_{ϕ_4}	$A_{v_1}^1$	$A_{v_2}^1$	$A_{v_3}^1$	$A_{v_4}^1$	$A_{v_1}^2$	$A_{v_2}^2$	$A_{v_3}^2$	$A_{v_4}^2$	B	C		B	C
t_{Φ_1}	1	1	0	0	0	0	0	0	0	0	0	0	0	0	✓	1	0
t_{Φ_2}	0	0	1	1	0	0	0	0	0	0	0	0	0	0	✓	2	0
$t_{v_1}^1$	1	1	0	1	0	1	1	1	1	0	0	0	1	0	✓	0	1
$t_{v_2}^1$	1	0	0	0	1	0	1	1	0	1	0	0	1	0	✗	0	0
$t_{v_3}^1$	0	1	1	1	1	1	0	1	0	0	1	0	1	0			
$t_{v_4}^1$	0	0	1	0	1	1	1	0	0	0	0	1	1	0			
$t_{v_1}^2$	1	1	0	1	1	0	0	0	0	1	1	1	1	2	0		
$t_{v_2}^2$	1	0	0	0	0	1	0	0	1	0	1	1	1	2	0		
$t_{v_3}^2$	0	1	1	1	0	0	1	0	1	1	0	1	1	2	0		
$t_{v_4}^2$	0	0	1	0	0	0	0	1	1	1	1	0	2	0			
t^1	0	0	0	0	1	1	1	1	0	0	0	0	0	0			
t^2	0	0	0	0	0	0	0	0	1	1	1	1	0	0			
$t_{v_1}^{\{1,2\}}$	0	0	0	0	0	1	1	1	0	1	1	1	0	0			
$t_{v_2}^{\{1,2\}}$	0	0	0	0	1	0	1	1	1	0	1	1	0	0			
$t_{v_3}^{\{1,2\}}$	0	0	0	0	1	1	0	1	1	1	0	1	0	0			
$t_{v_4}^{\{1,2\}}$	0	0	0	0	1	1	1	0	1	1	1	0	0	0			
t_C	0	0	0	0	0	0	0	0	0	0	0	0	0	1			

Figure 5: Relation and example set created for weighted circuit anti-monotone normalized satisfiability problem in Example 7.4.

- A tuple t_{Φ_j} for each Φ_j , containing 1 for each ϕ_i in its disjunction;
- k tuples t_n^m for each variable v_n , in which tuple t_n^m has (1) a value 1 for A_{ϕ_i} if v_n appears in ϕ_i , (2) a value 1 for every column corresponding to an attribute $A_{v_n'}^{m'}$ if both $m' = m$ and $n' \neq n$ or both $m' \neq m$ and $n' = n$ and (3) the value m in attribute B ;
- a tuple t^m , for all $m \leq k$ that has 1 in the attributes corresponding to the m -th copies of the variables;
- tuples $t_n^{\{m,m'\}}$, for each variable v_n , where $1 \leq m < m' \leq k$ with 1 for every attribute corresponding to a variable other than v_n ;
- a tuple with 1 in attribute C .

Consider the example set E , containing $k + 2$ tuples, appearing on the right-hand side of Figure 5. To understand how satisfying queries and satisfying assignments correlate, assume, to simplify, that all selections are equality constraints. Now, it is easy to observe that the only equality constraints that can appear in a satisfying query are of the form $A = 0$, where A is some attribute other than B or C .

A satisfying query for the example set will have 2 (in general, k) selections of the form $A_{v_n}^m = 0$ with different subscripts and different superscripts. Otherwise, we could not derive both of the first two positive examples, or we would derive the negative example, via t^1 or t^2 (if we use the same superscript) or via one of $t_n^{\{1,2\}}$ (if we use the same subscript). A satisfying condition must also contain a selection of the form $A_{\phi_i} = 0$, where $\phi_i \in \Phi_j$, for all j , since otherwise, one of the tuples t_{Φ_j} would agree with the selection condition, thereby deriving the negative tuple.

Thus, by careful consideration, it is possible to see that satisfying queries precisely correspond to satisfying assignments of weight k , e.g., the satisfying assignment assigning

precisely v_2 and v_4 to true, which satisfies the formulas ϕ_2 and ϕ_4 corresponds to the satisfying query

$$\pi_{B,C}(\sigma_{A_{\phi_2}=0}(\sigma_{A_{\phi_4}=0}(\sigma_{A_{v_2}^1=0}(\sigma_{A_{v_4}^2=0}(R))))) .$$

Finally, we consider Claim 3 of Theorem 7.3, i.e., NP-completeness of $\text{BSAT}(\mathcal{Q}^{\pi,\sigma(f)})$, when the number of examples (or even only the number of positive examples) is a constant. Membership in NP is immediate. To show completeness, we show that for any database $\mathcal{D}$ and example set E containing at most two positive examples, and arbitrarily many negative examples, we can define a database $\mathcal{D}'$ and example set E' , containing a constant number of examples, such that there is a satisfying query in $\mathcal{Q}_k^{\sigma(f)}$ for E over $\mathcal{D}$ if and only if there is a satisfying query in $\mathcal{Q}_{k+1}^{\pi,\sigma(f)}$ for E' over $\mathcal{D}'$. This will allow us to adapt the NP-hardness reduction used for Claim 1 of Theorem 4.5 (for $\text{BSAT}(\mathcal{Q}^{\sigma(f)})$), to show NP-hardness of $\text{BSAT}(\mathcal{Q}^{\pi,\sigma(f)})$, when the number of examples is a constant. We demonstrate this construction next.

EXAMPLE 7.5. Consider relation R_5 from Figure 1, and the example set E

	A	B	C
✓	1	2	2
✗	1	2	1
✗	1	1	2

We create a relation R' as follows:

A	B	C	X	Y
1	2	2	0	1
1	2	2	1	0
1	2	1	0	0
1	1	2	0	0

	SAT			BSAT			
	None	Sch	Exmp	None	Qry	Sch	Exmp
π	P	-	-	P	-	-	-
$\bowtie$	P	-	-	NPC	P $W[2]C$	P FPT	NPC
σ	P	-	-	NPC	P $W[2]C$	$\{=\} : P, \{\neq\} : NPC$ $\{=\} : FPT$	P FPT
$\sigma, \bowtie$	P	-	-	NPC	P $W[2]C$	$\{=\} : P, \{\neq\} : NPC$ $\{=\} : FPT$	NPC
π, σ	NPC	$\{=\} : P, \{\neq\} : NPC$ $\{=\} : FPT$	P $W[3]C$	NPC	P $W[3]C$	$\{=\} : P, \{\neq\} : NPC$ $\{=\} : FPT$	NPC
$\pi, \bowtie$	DP	P $W[1]H, co-W[1]H$	DP	DP	P $W[2]H, co-W[1]H$	P $W[1]H, co-W[1]H$	DP
$\pi, \sigma, \bowtie$	DP	$\{=\} : P, \{\neq\} : NPC$ $\{=\} : W[1]H, co-W[1]H$	DP	DP	P $W[3]H, co-W[1]H$	$\{=\} : P, \{\neq\} : NPC$ $\{=\} : W[1]H, co-W[1]H$	DP

Table 1: Table summarizing complexity results. Columns labeled “None” contain complexity results when none of the input is considered a constant or a parameter. Columns with “Qry,” “Sch,” “Exmp” contain results when the query size, schema size or number of examples, respectively, is considered a constant or parameter. Within each such column, the results for constant input appear on the first line (per operator), and the results for parameterized input appear on the second line (per operator). For cases in which the complexity is dependent on the type of selection conditions, these are listed explicitly. (To simplify the table, we included only the complexity results for “=” and “ $\neq$,” but not for “ $\leq$ ” or “ $\geq$.”) Note that we list parameterized complexity only when the complexity for the constant case is polynomial.

Note that this relation contains two tuples derived from the positive example, and one each from the negative examples. (Some changes are required to the construction if E contains more or less positive examples.)

We define E' as the example set

	X	Y
✓	0	1
✓	1	0
✗	0	0

Now, it is easy to see that a satisfying query of E over R_5 corresponds to one for E' over R' , e.g., as do Q and Q' in the following example

$$Q = \sigma_{B=2}(\sigma_{C=2}(R_5)) \quad Q' = \pi_{X,Y}(\sigma_{B=2}(\sigma_{C=2}(R'))).$$

8. CONCLUSION

This paper presented a complete study of the complexity of learning SPJ-queries from positive and negative examples. We considered a variety of query classes, with different input parameters. We compare the complexity when input parameters are constant, in the spirit of data complexity, and when performing parameterized complexity analysis. The latter, in particular, helps to delineate between cases that can, and cannot, be solved efficiently, in practice. Thus, this work lays the theoretical foundation for a practical system that can learn a candidate query from given example sets. Our complexity results are summarized in Table 1.

In the future we intend to extend this work in several important directions. First, we will consider additional operators, namely union and minus, as well as queries with aggregate operators. Second, we will study the learnability problem, i.e., determining the number of examples necessary to learn a specific given query.

Acknowledgments

The authors were partially supported by the Israel Science Foundation (Grant 879/16).

9. REFERENCES

- [1] Azza Abouzied, Dana Angluin, Christos Papadimitriou, Joseph M. Hellerstein, and Avi Silberschatz. Learning and verifying quantified boolean queries by example. In *PODS*, pages 49–60, New York, NY, USA, 2013. ACM.
- [2] Dana Angluin. Learning regular sets from queries and counterexamples. *Inf. Comput.*, 75(2):87–106, November 1987.
- [3] Dana Angluin. Queries and concept learning. *Machine Learning*, 2(4):319–342, April 1988.
- [4] Dana Angluin. Negative results for equivalence queries. *Machine Learning*, 5(2):121–150, July 1990.
- [5] Marcelo Arenas, Gonzalo I. Diaz, and Egor V. Kostylev. Reverse engineering sparql queries. In *Proceedings of the 25th International Conference on World Wide Web*, pages 239–249, 2016.
- [6] François Bancilhon. On the completeness of query languages for relational data bases. In *Mathematical Foundations of Computer Science*, pages 112–123, Berlin, Heidelberg, 1978. Springer Berlin Heidelberg.
- [7] Pablo Barceló and Miguel Romero. The complexity of reverse engineering problems for conjunctive queries. In *20th International Conference on Database Theory (ICDT 2017)*, 2017.
- [8] Angela Bonifati, Radu Ciucanu, and Aurélien Lemay. Learning Path Queries on Graph Databases. In *18th International Conference on Extending Database Technology (EDBT)*, Bruxelles, Belgium, March 2015.
- [9] Angela Bonifati, Radu Ciucanu, and Slawomir

- Staworko. Learning join queries from user examples. *ACM Trans. Database Syst.*, 40:24, 2016.
- [10] Sara Cohen and Yaacov Y. Weiss. Certain and possible xpath answers. In *Proceedings of the 16th International Conference on Database Theory, ICDT '13*, pages 237–248, New York, NY, USA, 2013. ACM.
- [11] Sara Cohen and Yaacov Y. Weiss. Learning Tree Patterns from Example Graphs. In Marcelo Arenas and Martín Ugarte, editors, *18th International Conference on Database Theory (ICDT 2015)*, volume 31 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 127–143, Dagstuhl, Germany, 2015. Schloss Dagstuhl–Leibniz-Zentrum fuer Informatik.
- [12] Sara Cohen and Yaacov Y. Weiss. The complexity of learning tree patterns from example graphs. *ACM Trans. Database Syst.*, 41(2):14:1–14:44, May 2016.
- [13] Rodney G. Downey and Michael R. Fellows. Fixed-parameter tractability and completeness I: basic results. *SIAM J. Comput.*, 24(4):873–921, 1995.
- [14] George H. L. Fletcher, Marc Gyssens, Jan Paredaens, and Dirk Van Gucht. On the expressive power of the relational algebra on finite sets of relation pairs. *IEEE Trans. Knowl. Data Eng.*, 21(6):939–942, 2009.
- [15] M.R. Garey, D.S. Johnson, and L. Stockmeyer. Some simplified NP-complete graph problems. *Theoretical Computer Science*, 1(3):237 – 267, 1976.
- [16] E Mark Gold. Complexity of automaton identification from given data. *Information and Control*, 37(3):302 – 320, 1978.
- [17] Martin Grohe. Parameterized complexity for the database theorist. *SIGMOD Rec.*, 31(4):86–96, December 2002.
- [18] Kouichi Hirata. On the hardness of learning acyclic conjunctive queries. In *11th International Conference on Algorithmic Learning Theory*, pages 238–251, Berlin, Heidelberg, 2000. Springer Berlin Heidelberg.
- [19] A. Nerode. Linear automaton transformations. *Proceedings of the American Mathematical Society*, 9(4):541–544, 1958.
- [20] Christos H. Papadimitriou and Mihalis Yannakakis. On the complexity of database queries. *Journal of Computer and System Sciences*, 58(3):407 – 427, 1999.
- [21] J. Paredaens. On the expressive power of the relational algebra. *Information Processing Letters*, 7(2):107 – 111, 1978.
- [22] Quoc Trung Tran and Chee-Yong Chan. How to conquer why-not questions. In *Proceedings of the 2010 ACM SIGMOD International Conference on Management of Data*, SIGMOD '10, pages 15–26, New York, NY, USA, 2010. ACM.
- [23] Quoc Trung Tran, Chee-Yong Chan, and Srinivasan Parthasarathy. Query by output. In *Proceedings of the 2009 ACM SIGMOD International Conference on Management of Data*, SIGMOD '09, pages 535–548, New York, NY, USA, 2009. ACM.
- [24] Meihui Zhang, Hazem Elmeleegy, Cecilia M. Procopiu, and Divesh Srivastava. Reverse engineering complex join queries. In *Proceedings of the 2013 ACM SIGMOD International Conference on Management of Data*, SIGMOD '13, pages 809–820, New York, NY, USA, 2013. ACM.

APPENDIX

A. PROOFS FOR W[3]-COMPLETENESS

PROPOSITION A.1. $\text{BSAT}(Q^{\sigma(f),\pi})$ is $W[3]$ -complete, when the query size bound is considered a parameter, for any $f \subseteq \Theta$.

PROOF. In order to show membership in $W[3]$, we present a reduction to the *weighted circuit satisfiability problem* with weft 3, a well known complete problem for $W[3]$ (see [13]). We briefly recall this problem.

In the weighted circuit satisfiability problem, we are given a Boolean circuit, with a single output gate, which is an AND gate. The circuit may contain both *small* gates of fan-in at most 2, and *large* AND/OR gates with arbitrary fan-in. The *weft* of a circuit is the maximum number of large gates on any path from an input to the output gate. Given a Boolean circuit, and a number k , the *weighted circuit satisfiability problem* is to determine whether there exists an input of Hamming weight at most k that is accepted by the circuit.⁵ For circuits of weft t , this problem is $W[t]$ -complete, given parameter k .

Let $\mathcal{D}$ be a database with a single relation R and E be an example set. We create a Boolean circuit, containing, for every attribute A in R , and every constant c associated with A in some tuple of R or in a negative example, variables (i.e., input gates) $x_{A\theta c}$, where $\theta \in f$. In addition,

- for each tuple t in R that can be a source of a positive example (i.e., that when projected yields a positive example), we create an AND-gate. For each condition $A\theta c$, with $\theta \in f$, such that t is disqualified when the selection $\sigma_{A\theta c}$ is applied, we connect variable $x_{A\theta c}$ to t using a NOT-gate.
- for each positive example $t^p \in E$, we create an OR-gate, which has as input all gates t such that t can be a source of t^p .
- for each tuple t in R that can be a source of a negative example, we create an OR-gate. For each condition $A\theta c$, with $\theta \in f$, such that t is disqualified when the selection $\sigma_{A\theta c}$ is applied, we connect variable $x_{A\theta c}$ to t .
- finally, all gates corresponding to positive examples, and to sources of negative examples feed into the output AND-gate.

Observe that the circuit constructed has weft 3. It is easy to see that a satisfying assignment of Hamming weight $k - 2$ will correspond immediately to a satisfying query of size k (by including both the selections that were assigned true, and the final projection). We demonstrate this construction in Example 5.4, when $f = \{\leq, \geq\}$. Note that input gates that are unreachable from the output gate are omitted from the figure.

In order to show $W[3]$ -hardness, we consider the more constrained *weighted circuit anti-monotone normalized satisfiability problem* with weft 3, which is also complete for $W[3]$ (see [13]). In this problem, all inputs are negated, and all paths from inputs to the root of the circuit tree pass through a large AND gate, a large OR gate, and another large AND gate. The problem is to determine whether there exists a

⁵The classic weighted circuit satisfiability problem is to determine whether there exists an input of Hamming weight precisely k . The flavor of the problem considered here is also $W[3]$ -complete.

	A_1^1	A_2^1	A_3^1	A_4^1	A_1^2	A_2^2	A_3^2	A_4^2	A_Φ	A'
t_{ϕ_1}	1	1	0	0	1	1	0	0	Φ_1	0
t_{ϕ_2}	1	0	1	0	1	0	1	0	Φ_1	0
t_{ϕ_3}	0	0	1	1	0	0	1	1	Φ_2	0
t_{ϕ_4}	1	0	1	0	1	0	1	0	Φ_2	0
t_1	1	1	1	1	0	0	0	0	0	0
t_2	0	0	0	0	1	1	1	1	0	0
$t_1^{\{1,2\}}$	0	1	1	1	0	1	1	1	0	0
$t_2^{\{1,2\}}$	1	0	1	1	1	0	1	1	0	0
$t_3^{\{1,2\}}$	1	1	0	1	1	1	0	1	0	0
$t_4^{\{1,2\}}$	1	1	1	0	1	1	1	0	0	0
$t_{A'}$	0	0	0	0	0	0	0	0	0	1

	A_Φ	A'
✓	Φ_1	0
✓	Φ_2	0
✓	0	1
✗	0	0

Figure 6: Construction for $W[3]$ -hardness in Proposition A.1 for the formula $((\neg v_1 \wedge \neg v_2) \vee (\neg v_3 \wedge \neg v_1)) \wedge ((\neg v_3 \wedge \neg v_4) \vee (\neg v_1 \wedge \neg v_3))$, with $p = 2$, as defined in Example 7.4.

satisfying assignment of weight precisely p . The constraint on the assignment weight is part of the problem definition. We will need to also take that into account when constructing a learning problem.

Let $\{x_1, \dots, x_n\}$ be a set of input variables. Let formulas of the form $\phi_j = \bigwedge_i \neg x_{j,i}$ represent large AND gates which conjunct negated variables. These formulas are one level deep. Let formulas of the form $\Phi_j = \bigvee_i \phi_{j,i}$ represent large OR gates which disjunct formulas which themselves are one level deep conjunctions with all negative variables. These formulas are DNF formulas with only negative variables. Finally, let $\Psi = \bigwedge_i \Phi_{j,i}$ be a conjunction of formulas of the above form. The resulting circuit or formula is anti-monotone and normalized, with weft 3, and finding a satisfying assignment of weight p is $W[3]$ complete, when p is a parameter.

We next show how to convert this weighted satisfiability problem Ψ a query satisfiability problem $\text{SAT}(\mathcal{Q}_k^{\sigma(f), \pi})$, when the query size k is parameterized. We need to describe what each of the variables represents, what each gate in circuit represents, and how we require a satisfying query to represent a satisfying assignment of the right size.

In this construction, a weight parameter p , will correspond to a query size parameter of $k = p + 2$. (p selections, 1 for the relation with no selections, and 1 for the projection.)

The construction will have the following properties:

- Each formula Φ_i will be represented by a positive example e_i^+ .
- Each formula ϕ (as part of a disjunction of Φ_i) will be converted into a tuple which can satisfy the positive example e_i^+ .
- Each variable x_i will be represented by a possible selection.

Additional examples will be added in order to allow only appropriate selections, and require the right number of them. An assignment of weight p for Ψ will then correspond to a query with p selections, i.e., a query of size $p + 2 = k$. A satisfying assignment will clearly correspond to a satisfying query.

Let Ψ be a formula as above, for which we want to find a satisfying assignment with weight p . We build the following database $\mathcal{D}_\Psi$, with a single relation R , which has the following attributes:

- p attributes $A_i^1, \dots, A_i^p$ for each variable x_i in Ψ . Intuitively, selections on an attribute A_i^j can be interpreted as saying that x_i is the j th positive variable in an assignment. These attributes will be projected out.
- An attribute A_Φ , which represents which positive example would be satisfied by selecting this tuple.
- An attribute A' which will be used to prevent selections on A_Φ . Selections will also be forbidden on this attribute, i.e., such selections will create a query that is not satisfying.

The rest of the construction will ensure that the only selections that will possibly derive a satisfying query will be on the attributes A_i^j , and only those which force their value to be 0.

The relation R will contain the following tuples: (In all tuples, all values not specifically mentioned are 0.)

- Each formula ϕ_j , which is a subformula of Φ_i , will be represented by a tuple t_{ϕ_j} where attributes representing variables in ϕ will get the value 1, and the attribute A_Φ will get the value Φ_i .
- For each $1 \leq j \leq p$, we add a tuple t_j where all attributes A_i^j get the value 1. These will be used to ensure that we choose p selections.
- For each variable x_m and each pair $1 \leq j_1 \neq j_2 \leq p$, we add a tuple $t_m^{\{j_1, j_2\}}$ in which all attributes $A_i^{j_1}$ and $A_i^{j_2}$ get the value 1, with the exception of the attributes $A_m^{j_1}$ and $A_m^{j_2}$ which receive the value 0. This will be used to ensure that we will not select the same variable twice.
- A tuple $t_{A'}$ where the attribute A' receives a value of 1. This will be used to prevent selections directly on A_Φ .

Finally, the example set will contain:

- A positive example $\{A_\Phi : \Phi_i, A' : 0\}$ for each subformula Φ_i in Ψ .
- A positive example $\{A_\Phi : 0, A' : 1\}$.
- A negative example $\{A_\Phi : 0, A' : 0\}$.

This construction is demonstrated in Figure 6.

We now need to show that the reduction is valid, i.e., that there is a satisfying query Q with a size no more than

$k = p + 2$ if and only if Ψ has a satisfying assignment with a weight of exactly p .

Consider a truth assignment μ that assigns $x_{s_1} \dots x_{s_p}$ to true, and all other variables to false. Then by construction, the query $Q = \pi_{A_{\Psi}, A'} \sigma_{A_{s_1}^1=0} \dots \sigma_{A_{s_p}^p=0} R$ will satisfy the example set E over $\mathcal{D}$ if and only if μ satisfies Ψ . This holds because by construction selections remove tuples, corresponding to a ϕ clause in Ψ which are dissatisfied by positive variables. A positive example is satisfied by making sure at least one of its possible source tuples remains, which corresponds to a Φ clause being satisfied because at least one of its ϕ clauses is not dissatisfied. The size of Q is exactly $k = p + 2$ as required. It immediately follows that if Ψ has a satisfying assignment, then there is a query satisfying E over $\mathcal{D}$.

In the other direction, suppose Q satisfies E over $\mathcal{D}$. If Q has the form above, and each of the attributes represent different variables, satisfaction of Ψ is immediate. We also know from construction that Q may only include comparisons where attributes must equal 0. (Or equivalently, attributes must not be equal to 1, or not more than 0.) Suppose, on the other hand, that Q has a different form. If Q does not include a projection, the schema returned by Q and the schema of the example set will differ, preventing satisfaction. Since Q must include a projection operator, it may include at most p selections. Suppose Q includes less than p selections. Then there is some j , for which none of the attributes A_i^j are selected upon. It immediately follows that Q cannot satisfy, as it will return a negative example (using t_j). We therefore know that Q must include exactly p selections, as $p + 1$ selections would already make the query too big. We immediately see that for each $1 \leq j \leq p$, there is exactly one attribute A_i^j upon which we select. Finally, suppose there is some $j_1 \neq j_2$ and some i such that Q selects upon both $A_i^{j_1}$ and $A_i^{j_2}$. In that case, one of the tuples described by the third bullet above will prevent satisfaction by Q , as each j can only have a single selection of the form A_i^j . It follows that if Q satisfies E over $\mathcal{D}$, it must have the form of the query mentioned above, which must correspond to a satisfying assignment over Ψ , as required. $\square$

PROPOSITION A.2. $\text{SAT}(Q^{\pi, \sigma(f)})$ is $W[3]$ -complete when the number of examples is a parameter.

PROOF. To show membership in $W[3]$, the same reduction used to prove Proposition A.1 cannot be applied, as we no longer have a limit on the size of the query. Hence, we provide a different modeling of the problem at hand as a Boolean circuit with weft 3. Let $\mathcal{D}$ be a database with a single relation R and E be an example set with e examples. We create a Boolean circuit with input gates x_t for each tuple t in R that is a possible source of a positive example from E . In addition,

- for each selection of interest $A\theta c$, where A is an attribute in R and $\theta \in f$, we create an AND-gate. An input gate x_t that is a possible source of a positive tuple is connected to this gate via a NOT-gate if t is disqualified by $A\theta c$.
- for each positive example $t^p \in E$, we create an OR-gate, which has as input all gates x_t such that t can be a source of t^p .

- for each tuple t in R that can be a source of a negative example, we create an OR-gate. For each condition $A\theta c$, with $\theta \in f$, such that t is disqualified when the selection $\sigma_{A\theta c}$ is applied, we connect variable $x_{A\theta c}$ to t .
- finally, all gates corresponding to positive examples, and to sources of negative examples feed into the output AND-gate.

Observe that the circuit constructed has weft 3. We demonstrate this construction in Figure 7, when $f = \{\leq, \geq\}$. Note that this circuit is easily derived from the circuit in Figure 3 by changing the direction of the arrows from the input variables to the gates representing selections (and then re-assigning which gates are input and which are AND-gates appropriately).

It is easy to see that there is a query satisfying the example set E if and only if there is a satisfying assignment of Hamming weight at most e , as the source tuples for the positive examples will be assigned true (and it is sufficient to have e such source tuples), and the query itself can be read off the circuit by seeing which gates corresponding to selection have the value true.

In order to show $W[3]$ -hardness, we again consider the *weighted circuit anti-monotone normalized satisfiability problem* with weft 3, using the same notation used for Proposition A.1. Recall that we have a formula Ψ , which is a conjunction of formulas called Φ , which are disjunctions of formulas called ϕ which are conjunction of negative literals.

In this construction, a weight parameter p , will correspond to a parameter $e = p + 2$, bounding the number of examples. (p positive examples corresponding to the formula, 1 negative example, and an additional positive example to prevent unwanted selections.)

The construction will have the following properties:

- Each formula Φ_i will be represented a tuple forbidden by the negative example.
- Each formula ϕ (as part of a disjunction of Φ_i) will be converted into a possible selection, which can remove the forbidden tuple which represents Φ_i .
- Each variable x_i will be represented by a tuple which can be removed by any of the selections representing formulas containing the variable.

Additional examples will be added in order to allow only appropriate selections. An assignment of weight p for Ψ will then correspond to a query which does not remove p chosen tuples. A satisfying assignment will clearly correspond to a satisfying query.

Let Ψ be a formula as above, for which we want to find a satisfying assignment with weight p . We build the following database $\mathcal{D}_{\Psi}$. $\mathcal{D}_{\Psi}$ will contain a single relation R with the following attributes.

- An attribute A_{ϕ_i} for each conjunction ϕ_i in Ψ . These attributes will be projected out.
- p attributes $A_i^1, \dots, A_i^p$ for each variable x_i in Ψ . Intuitively, selections on an attribute A_i^j can be interpreted as saying that x_i is the j th positive variable in an assignment. These attributes will also be projected out.
- An attribute B .
- An attribute C which will be used to prevent selections on B . Selections will also be forbidden on this attribute.

R			
	A	B	C
t_1	1	2	3
t_2	1	3	3
t_3	1	4	4
t_4	1	3	4

E			
		A	C
✓	t_p	1	3
✗	t_n	1	4

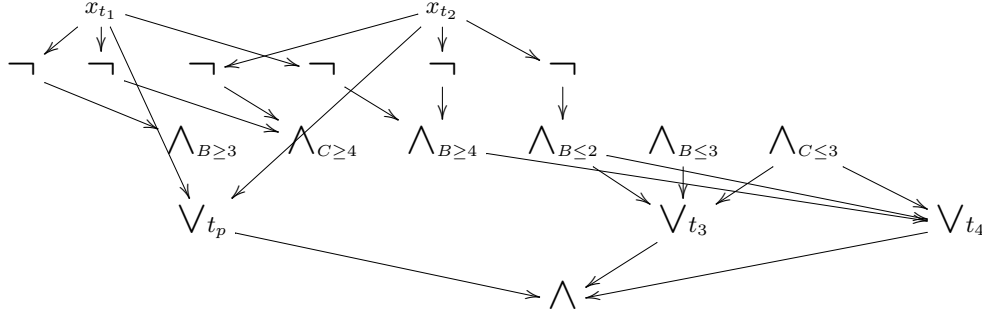


Figure 7: Reduction to $W[3]$ for Proposition A.2.

The only permitted selections will be on the attributes A_{ϕ_i} and A_i^j , and only those which force their value to be 0.

The relation R will contain the following tuples: (In all tuples, all values not specifically mentioned are 0.)

- Each formula Φ , represents a forbidden tuple, which will have the value 1 in each attribute A_{ϕ_i} when ϕ_i is a subformula of Φ .
- For each $1 \leq j \leq p$, and for each variable x_i , we add a tuple where the attribute B gets the value j . An attribute A_{ϕ_k} gets the value 1 if x_i is in ϕ_k . An attribute A_m^j gets the value 1 when $i \neq m$. An attribute A_i^m gets the value 1 when $m \neq j$. Intuitively, these tuples state that the j th variable is x_i .
- For each $1 \leq j \leq p$, we add a tuple with all attributes A_i^j getting the value 1. This will ensure we choose p satisfying tuples.
- For each variable x_m and each pair $1 \leq j_1 \neq j_2 \leq p$, we add a tuple with the attributes $A_i^{j_1}$ and $A_i^{j_2}$ getting the value 1, with the exception of the attributes $A_m^{j_1}$ and $A_m^{j_2}$ which receive the value 0. This ensures we don't select the same variable twice.
- A tuple where the attribute C receives a value of 1. This will be used to prevent selections directly on A_Φ .

Finally, the example set will contain:

- A positive example $\{B : j, C : 0\}$ for each $1 \leq j \leq p$.
- A positive example $\{B : 0, C : 1\}$.
- A negative example $\{B : 0, C : 0\}$.

This construction is demonstrated in Example 7.4.

We now need to show that the reduction is valid, i.e., that there is a query satisfying E over $\mathcal{D}_\Psi$, if and only if Ψ has a satisfying assignment with a weight of exactly p .

Consider a query q which satisfies E over $\mathcal{D}_\Psi$. It is clear that q must have the following properties:

- For each $1 \leq j \leq p$, there is a selection on some attribute A_i^j . (Otherwise negative example is not satisfied.)
- For each $1 \leq j \leq p$, there are no selections on more than one attribute A_i^j . (Positive example $\{B : j, C : 0\}$)
- No two $1 \leq j_1 \neq j_2 \leq p$ share the same i by selecting on both $A_i^{j_1}$ and $A_i^{j_2}$. (Otherwise negative example is not satisfied.)

These selections completely determine which tuples satisfy which positive examples. These tuples correspond with variables in the weighted circuit satisfaction problem. A maximal selection can be found from such a base, by including all possible selections which do not prevent satisfaction of the positive examples. This provides a correspondence between assignments and queries.

The correspondence between satisfying assignments and satisfying queries is immediate, as satisfying Φ clauses by choosing a satisfying subformula ϕ , corresponds to choosing selections to remove unwanted tuples, while the variables prevent satisfaction of some ϕ clauses, as positive examples prevent usage of some selections. $\square$